N6NU SDR WSJT-X / QMAP Bridges

A family of Windows applications by **Andreas Junge, N6NU** <andreas@n6nu.org> that adapt commodity SDR hardware (HackRF, RTL-SDR, SDRplay, AirSpy, FunCube Pro+, Malachite-style sound-card receivers, ADALM-Pluto / Pluto+) into wideband Q65 (QMAP) feeds and WSJT-X-friendly RX / TX paths for VHF and microwave EME work.

Document generated 2026-05-04. Eight products distributed as free Windows installers under GPLv3.

1. The family at a glance

All eight bridges share one C++/Qt6 codebase (`bridge-core`) for the UI, audio routing, Linrad/QMAP wire format, FFT, SSB demod, CAT server, and so on. Each app is a thin radio-specific wrapper over that core.

App (binary name)	Role	SDR hardware	Native ADC	Version
`hackrf-wsjtx-bridge` TX+RX	Full half-duplex transceiver. WSJT-X drives this as the rig.	HackRF One	8-bit	v1.0.1
`hackrf-rx-bridge` RX	Wideband observer next to your real rig.	HackRF One	8-bit	v1.0.1
`rtlsdr-rx-bridge` RX	$25 dongle wideband observer + Q65 path.	RTL-SDR (R820T/T2, R820T2 V3+)	8-bit	v1.0.2
`sdrplay-rx-bridge` RX	14-bit-end-to-end wideband observer (RSPduo / RSPdx / RSP1A).	SDRplay RSP family	14-bit	v1.0.5
`airspy-rx-bridge` RX	AirSpy R2 wideband observer, low-noise EME-targeted gain presets.	AirSpy R2 (HF+ planned)	12-bit	v0.99.3
`iq-rx-bridge` RX	Generic sound-card-IQ feeder + FunCube HID CAT, with built-in SSB demod for IQ-only sources.	Malachite, FunCube Pro+, generic IF tap on a USB sound-card	16- to 24-bit (sound card)	v1.0.4
`pluto-rx-bridge` RX	RX-only Pluto / Pluto+ over libiio (Ethernet or USB-RNDIS).	ADALM-Pluto, Pluto+	12-bit (AD9361/AD9363)	v1.0.0
`pluto-wsjtx-bridge` TX+RX	Full transceiver against an ADALM-Pluto / Pluto+. WSJT-X PTT-driven.	ADALM-Pluto, Pluto+	12-bit (AD9361/AD9363)	v1.0.0

2. Common features (every bridge in the family)

2.1 RX signal chain

- **Wideband IQ → Linrad Raw16 UDP** for QMAP. Default target `127.0.0.1:50004`; use `--linrad-udp-target 192.168.0.255` for subnet broadcast so all QMAP instances on the LAN receive the stream (see §5.8). 96 kHz IQ output rate with the SDR's full passband resampled into Linrad packets (1416 bytes, 348 IQ pairs each, the QMAP-expected format including the $(-1)^n \cdot \text{conj}()$ transform QMAP needs for its FFT-direction quirk).
- **SSB demod → 48 kHz audio** for WSJT-X RX. Hilbert-phasing USB/LSB selection at audio rate; sideband tracks the mode reported on the WSJT-X UDP / CAT channels.
- **Spectrum / waterfall display** with span auto-matching the real IQ rate (no more hard-coded 2 MHz default). Toggle via View → Show spectrum waterfall (Ctrl+W); persisted across runs.
- **Front-end overload indicator** on radios that report it.

2.2 Audio routing

- **RX audio out** to a Windows audio device (typically VB-Cable Line 1) so WSJT-X picks it up as a microphone-equivalent input.
- **TX audio in** (TX bridges only) from a Windows audio device, typically the WSJT-X output side of VB-Cable.
- **RX audio gain** in 1 dB steps, persisted.
- Mode selector follows WSJT-X-reported mode (USB / LSB / PKTUSB / PKTLSB).

2.3 Network protocols

- **WSJT-X UDP listener** (default port 2237) reads the "Status" message: dial frequency, mode, transmitting state. Configurable port (multi-instance).
- **Linrad TCP parameter server** (default 49812). Used by MAP65 / Linrad-the-app for the parameter handshake; **QMAP does not use this port** (no `CONNPORT` key in `qmap.ini`) but it must still be unique per bridge instance to avoid bind collisions. The bridge always binds it.
- **Linrad UDP target** (default 50004) is the QMAP listen port. Per-instance unique (50005, 50006, …) for multi-band ops. The target address defaults to `127.0.0.1` (localhost) but **should be set to the subnet broadcast address** (e.g. `192.168.0.255`) when QMAP runs on a different machine — see §5.8.
- **TCI WebSocket server** (default port 40001). Expert Electronics Thin Client Interface. WSJT-X can connect natively via TCI for frequency control, PTT, mode, and TX audio. Opt-in via `--tci-port`. See §5.11.
- **rigctld-compatible CAT server** (Hamlib NET rigctl TCP). **Opt-in** on every RX-only bridge; always on for the TX bridges. Family port sequence: 4532 (real Hamlib rigctld), 4533 (hackrf-wsjtx-bridge), 4534 (pluto-rx-bridge), 4535 (rtlsdr-rx-bridge), 4536 (pluto-wsjtx-bridge), 4537 (hackrf-rx-bridge), 4538 (sdrplay-rx-bridge), 4539 (airspy-rx-bridge), 4540 (iq-rx-bridge).
- **Live source indicator** in the window title: `— UDP` / `— UDP (CAT idle)` / `— CAT (n)`, updated every second. Tells you at a glance which path is currently driving the dial. The status panel also shows CAT and TCI client counts (green = connected, yellow = listening, absent = not started).

2.4 GUI / lifecycle

- **Multi-instance support** via `--instance <name>`. Namespaces the INI file, window title, and taskbar entry. Run two bridges side-by-side for two-band ops without their settings clobbering each other.
- **Manual frequency override** decouples the bridge dial from WSJT-X for static observation of beacons / sounders. Reset button re-couples.
- **Transverter offset** in MHz, signed. SDR tunes to `dial + offset`; GUI and QMAP keep showing the dial. e.g. `--transverter-offset -10224` for 10368 MHz operation through a 144 MHz IF transverter.
- **Settings persistence** in `%APPDATA%\Roaming\n6nu\<App Name>.ini` (or `<App Name> - <instance>.ini` when `--instance` is set).
- **Optional** `--console` flag on Windows pops a debug console window so you can watch `[Stats]`, freq update, and error log lines live.
- **Headless mode** via `--no-gui`.

2.5 Diagnostics

- 5-second `[Stats]` line on stderr / console with: sample rate seen, peak |IQ| in dBFS, ms since last freq update, CAT client count (when CAT is on).
- Boot banner with version + build timestamp.
- Per-event log lines: `[WSJT-X UDP] dial → X Hz`, `[CAT rigctld] freq → X Hz`, etc.

3. Device-specific features

Bridge	Unique features & controls
`hackrf-wsjtx-bridge`	Full TX+RX. SsbModulator with cos/sin upconvert at the 1.5 kHz IF offset (HackRF TX is at IF, not LO). HackRF gain stages: TX VGA 0–47, RX LNA 0–40, RX VGA 0–62, AMP enable. Bias-tee toggle (used as transverter TX-keying line on bench setups). PTT pre-fill audio injection on key-down. CAT (rigctld) on TCP 4533 always on (it's the rig WSJT-X talks to). Custom MainWindow with PTT button.
`hackrf-rx-bridge`	RX-only. Same HackRF gain stages as above, no TX path. RX-only sibling to the full transceiver for "observe with dongle while real rig handles QSO" use cases. Opt-in rigctld CAT on TCP 4537.
`rtlsdr-rx-bridge`	Tuner gain snapped to nearest hardware-supported step. Auto-gain (R820T2 internal AGC) toggle. RTL2832 IF AGC toggle. Bias-tee (RTL-SDR.com V3+ only). Direct-sampling I or Q mode for HF, plus auto-switch <25 MHz. `--device-index` for multiple dongles. PPM correction. Opt-in rigctld CAT on TCP 4535.
`sdrplay-rx-bridge`	14-bit IQ end-to-end through the wire format and waterfall — higher dynamic range than 8-bit family members. RSPduo single-tuner on Tuner A. RSPdx antenna A/B/C selector. Per-model bias-T, notches, HDR (model-specific knobs). LNA-state + IF gain reduction (gr-db) controls. Structural changes (sample rate, IF mode) require a full Uninit → re-apply → Init cycle — bridge handles automatically. Opt-in rigctld CAT on TCP 4538.

Bridge	Unique features & controls
`airspy-rx-bridge`	AirSpy R2's three gain modes: *Linearity* presets (low-IMD on crowded bands), *Sensitivity* presets (low-noise for VHF EME, bridge default), and *Manual* (LNA / Mixer / VGA stages 0–15 each). Native rate selector populated from libairspy (R2 supports 2.5 / 10 Msps). LNA + Mixer AGC toggle. Bias-T toggle (no-op on R2; HF+ forward-compat). Opt-in rigctld CAT on TCP 4539.
`iq-rx-bridge`	Generic Windows sound-card IQ feeder for Malachite-DSP, FunCube Pro+, FlexRadio DAX-IQ, generic IF taps. Auto-detects the actual negotiated sample rate (FCD Pro+ V2 outputs 192 kHz natively even when 96 kHz is requested) and uses that for the QMAP / waterfall chain. **Built-in SSB demod** pushes 48 kHz mono audio to VB-Cable Line 1 so WSJT-X decodes FT8/FT4 directly — required for FunCube Pro+ and any IQ-only source; toggleable in Settings for radios that already produce their own SSB audio (Malachite-DSP). Two CAT layers cooperate: the radio-side *CAT controller* (Generic / FunCube HID) tunes the dongle, and the opt-in WSJT-X-side *rigctld CAT server* on TCP 4540 receives Doppler corrections and chains them through. FCD Pro+ gain stages (LNA, mixer, IF1, IF2, IF3 with enhancement, IF filter) and bias-T over HID.
`pluto-rx-bridge`	libiio context over the network backend by default (`ip:HOST`) or USB-RNDIS or local. Settings UI for the libiio URI. AD9361 gain modes: manual (−3..71 dB), fast-attack, slow-attack (default for weak signal), hybrid. RF bandwidth (200 kHz – 56 MHz) and complex-IQ rate (2.083 – 61.44 Msps) configurable. PPM trim against the Pluto+'s 0.5 ppm VCTCXO. **Auto-reconnect** on Pluto reboot / cable bump / network glitch (5 s libiio timeout to short-circuit Windows' default 2-hour TCP keepalive; exponential backoff retry up to 30 s; setters are mutex-protected so freq/gain updates stay coherent across the disconnect window). Opt-in rigctld CAT on TCP 4534 with auto-detect UDP-mute when a CAT client connects.
`pluto-wsjtx-bridge`	All RX features of pluto-rx-bridge, PLUS the AD9361 TX path. Direct-conversion: Pluto's TX_LO is the dial freq, the AD9361 takes baseband I/Q at TX_LO directly. WSJT-X audio captured from VB-Cable → SsbModulator's Hilbert phaser (USB/LSB) at 48 kHz → SoXr resampler I/Q to the Pluto's IQ rate, scaled to int16 for the DAC. TX attenuation 0 to −89.75 dB and TX RF bandwidth controllable from the Settings dialog (hot-swap on Apply). WSJT-X UDP `transmittingChanged` AND CAT `\set_ptt` / `T VFOA n` both drive half-duplex `startTx / stopTx` with `modulator.reset()` between bursts. TX silenced on key-up (TX_LO powered down + max attenuation; no residual carrier between transmissions). Cross-INI seed from `pluto-rx-bridge` on first launch + *Discover* button (libiio scan) for zero-config setup. **Verified with 2-way Q65 contacts on an IC-705 (2026-05-04).**

4. GitHub project locations

Repo	Type	URL
`n6nu/HackRF-WSJTX-Bridge`	Binary release (full TX+RX)	github.com/n6nu/HackRF-WSJTX-Bridge
`n6nu/HackRF-RX-Bridge`	Binary release (RX-only HackRF)	github.com/n6nu/HackRF-RX-Bridge

Repo	Type	URL
`n6nu/rtlsdr-rx-bridge`	Binary release (RX-only RTL-SDR)	github.com/n6nu/rtlsdr-rx-bridge
`n6nu/sdrplay-rx-bridge`	Binary release (RX-only SDRplay)	github.com/n6nu/sdrplay-rx-bridge
`n6nu/airspy-rx-bridge`	Binary release (RX-only AirSpy)	github.com/n6nu/airspy-rx-bridge
`n6nu/iq-rx-bridge`	Binary release (sound-card IQ + FunCube CAT) Formerly `n6nu/malachite-rx-bridge`; GitHub redirect kept old links alive.	github.com/n6nu/iq-rx-bridge
`n6nu/pluto-rx-bridge`	Binary release (RX-only Pluto / Pluto+)	github.com/n6nu/pluto-rx-bridge
`n6nu/pluto-wsjtx-bridge`	Binary release (full TX+RX Pluto)	github.com/n6nu/pluto-wsjtx-bridge

Each binary repo's README has a "Latest release" link to the `.exe` installer for that bridge, plus a `RELEASE_NOTES.md` with the per-version changelog. Releases are also tagged (`v0.99.x`, `v1.0.x`, etc.) so you can pull a known-good older build if a new release misbehaves.

5. Setup gotchas

5.1 Microsoft Visual C++ Redistributable (every bridge)

> **Most common reason a bridge crashes at startup with no useful error.** Qt 6.8 (which all bridges link against) hard-requires the **Microsoft Visual C++ 2015–2022 Redistributable (x64)**. On a clean Windows 10/11 box without Visual Studio Build Tools or any other VC++-using app, the bridge .exe fails to load and Windows shows the generic "*<App> has stopped working*" dialog with no specifics.
>
> **Fix:** install `VC_redist.x64.exe` once on the target box (free, ~25 MB) and relaunch the bridge.
> N.B. the bridge installers do not currently bundle this. It will be added to the family-wide installer in a future release.

5.2 VB-Audio Virtual Cable bit depth

> **VB-Cable must be set to 16-bit, NOT 24-bit** in Windows Sound Properties. WSJT-X negotiates 24-bit by default on some Windows configurations, which the bridge's audio code falls back from to "preferred" 32-bit float and then WSJT-X reads the result as silence.
>
> **Fix:** Right-click VB-Cable Input / Output in Windows Sound → Properties → Advanced → **16 bit, 48000 Hz**. Apply on both sides of the cable.

5.3 SDRplay API package

The `sdrplay-rx-bridge` needs the **SDRplay API v3.15** (or newer 3.x) installed. Free download from sdrplay.com/api. The bridge will report "*No SDRplay receivers found*" at startup if the API isn't installed even when the device is plugged in and recognised by Windows.

5.4 RTL-SDR / HackRF Zadig WinUSB binding

RTL-SDR dongles and HackRF One both need the WinUSB driver bound to them via Zadig before `librtlsdr` / `libhackrf` can talk to them. Symptom: bridge logs "*hackrf_open failed: HackRF not found*" or "*librtlsdr_open() failed: -1*" even though Windows Device Manager shows the device.

The `rtlsdr-rx-bridge` installer optionally launches Zadig at install time. The HackRF installers don't bundle it — install Zadig separately and bind WinUSB to the HackRF One device listed in Zadig's dropdown.

5.5 ADALM-Pluto / Pluto+ driver bundle

For the two Pluto bridges, install the **Analog Devices PlutoSDR-M2K Windows driver bundle** from analog.com. This installs the USB-RNDIS / CDC-ACM / DFU drivers Windows needs to recognise the Pluto. Without them, the Pluto comes up as a USB Mass Storage device only (D: drive) and you can't talk to it via libiio.

The bridges bundle **libiio v0.26** (LGPLv2.1) and its transitive deps (`libusb-1.0` , `libxml2` , `libserialport-0`) inside the installer, so you do *not* need a separate libiio install — only the ADI driver bundle for the USB enumeration.

5.6 Pluto firmware variants and IP address

Default URI is `ip:192.168.2.1` — the stock Pluto USB-RNDIS address. **Pluto+** on Ethernet typically gets a DHCP address (e.g. `ip:192.168.0.148`). Change in Settings → Host URI or pass `--host ip:HOST` on the command line. Other libiio backends (`usb:` , `local:`) also accepted.

Pluto+ and Pluto running F5OEO Tezuka firmware identify themselves as stock `Analog Devices PlutoSDR Rev.C (Z7010-AD9363A)` via libiio, which is fine — the bridge treats them all uniformly. AD9361 unlock (extends 70 MHz – 6 GHz, vs AD9363's stock 325 MHz – 3.8 GHz) is detected via the device's `frequency_available` attribute — the bridge doesn't enforce its own software clamp.

5.7 Pluto TX requires a physical antenna

> For `pluto-wsjtx-bridge` : the Pluto's **TX1 SMA jack must have an antenna or load attached** before the bench-test tone (or any TX session) is detectable on a nearby receiver. Internal trace coupling alone is below the noise floor of most VHF/UHF receivers at >1 m distance. Symptom: bridge logs "*[Pluto] TX worker started*" with no `iio_buffer_push failed` errors, but receivers nearby see no signal change — looks identical to "TX is broken". Always check the antenna first.

5.8 Linrad TCP vs UDP — QMAP only uses UDP

Defaults: TCP 49812 (Linrad parameter server), UDP 50004 (QMAP listen). **QMAP doesn't read the TCP port** — `qmap.ini` has no `CONNPORT` key. The TCP port is for MAP65 / Linrad-the-app and the bridge still binds it,

so the port still has to be unique per bridge instance to avoid bind collisions. For multi-instance setups: bridge #1 49812/50004, bridge #2 49813/50005, etc.

5.8.1 UDP target — localhost, unicast, or broadcast

The `--linrad-udp-target` flag (and the *Linrad UDP target* field in Settings) controls where IQ packets are sent. Three topologies:

Topology	Flag	Use case
Local	`--linrad-udp-target 127.0.0.1` *(default)*	Bridge and QMAP on the same machine.
Unicast	`--linrad-udp-target 192.168.0.50`	Bridge on one machine, QMAP on another. Only that one QMAP receives the stream.
Broadcast	`--linrad-udp-target 192.168.0.255`	Bridge on one machine, *all* machines on the subnet can run QMAP and receive the stream. This is the recommended setup for remote SDR operation (bridge at the dish, decode clients in the office).

> **Which broadcast address?** Use the broadcast address for your subnet. For a typical `192.168.0.x` network that's `192.168.0.255`. For `10.0.0.x` it's `10.0.0.255`. If you're not sure, check `ip addr` (Linux) or `ipconfig` (Windows) — look for *Broadcast* in the interface details.

Each bridge can use a different UDP port for its QMAP stream (`--linrad-udp-port`), and each QMAP instance listens on its own port. For multi-band setups you might have:

```
hackrf-rx-bridge  --linrad-udp-target 192.168.0.255 --linrad-udp-port 50004
rtlsdr-rx-bridge  --linrad-udp-target 192.168.0.255 --linrad-udp-port 50005
```

Then QMAP #1 listens on 50004, QMAP #2 on 50005, each decoding a different band.

5.9 CAT server vs WSJT-X UDP path

The RX-only bridges default to **CAT off** (passive UDP-observer mode) so they don't get in the way of WSJT-X driving a real radio. Turning on CAT lets the bridge BE the radio, useful for WSJT-X Doppler tracking (which is implemented at the CAT layer — **WSJT-X with Rig=None won't apply Doppler correction**, no matter which Doppler mode is selected; without a rig to command, the corrections aren't applied to the published dial frequency).

If you want Doppler tracking on an RX-only bridge: enable CAT in Settings, restart the bridge, and set WSJT-X → Settings → Radio → **Hamlib NET rigctl**, Network Server `127.0.0.1:<port>` matching the bridge's CAT port. Test CAT may take ~10 s to turn green — that's WSJT-X's capability-probe sequence and is a one-time wait. Once green, freq updates over CAT happen sub-millisecond.

5.10 Multi-instance: instance namespace + port juggling

To run two bridges side-by-side (e.g. two RTL-SDRs feeding two WSJT-X / QMAP pairs on different bands):

```
rtlsdr-rx-bridge --instance 2304 --device-index 0 --wsjtx-port 2237 \
    --linrad-tcp-port 49812 --linrad-udp-port 50004 \
    --linrad-udp-target 192.168.0.255 --tci-port 40001
rtlsdr-rx-bridge --instance 2320 --device-index 1 --wsjtx-port 2238 \
    --linrad-tcp-port 49813 --linrad-udp-port 50005 \
    --linrad-udp-target 192.168.0.255 --tci-port 40002
```

The two bridges get separate INI files (`RTL-SDR RX Bridge - 2304.ini` and `RTL-SDR RX Bridge - 2320.ini`), separate window titles, separate taskbar entries.

5.11 TCI (Thin Client Interface) for remote WSJT-X

TCI is a WebSocket protocol (Expert Electronics) that lets WSJT-X connect to the bridge as if it were a networked radio. It provides:

- **Frequency / mode control** — VFO A, mode (USB/LSB/CW/etc.)
- **PTT** — TX on/off (TX bridges)
- **TX audio** — WSJT-X sends audio over the WebSocket (TX bridges)
- **SMeter / IQ stream** — signal strength and IQ data for client-side display

Enable it with `--tci-port 40001` (or any free port). WSJT-X connects in Settings → Radio → **TCI Client**, entering the bridge's IP and port.

TCI coexists with CAT and Linrad UDP — all three can run simultaneously:

- **CAT (rigctld TCP 4533+)** — WSJT-X's original remote control method. Opt-in with `--cat` .
- **TCI (WebSocket 40001+)** — native WSJT-X remote control plus TX audio. Opt-in with `--tci-port` .
- **Linrad UDP (50004+)** — always on, streams IQ to QMAP.

The bridge status panel shows all active connections: CAT clients (green = connected, yellow = listening) and TCI clients (same). Inactive protocols don't appear.

6. Family port-and-key reference

Default port	Used by
`UDP 2237`	WSJT-X UDP Status broadcasts (bridge listens)
`UDP 50004`	Linrad target / QMAP listen (bridge sends; use `--linrad-udp-target` to set destination)
`TCP 49812`	Linrad parameter server (bridge listens; MAP65 / Linrad use it)
`WS 40001`	TCI WebSocket server (bridge listens; WSJT-X connects)
`TCP 4532`	Hamlib rigctld default (real rigs — reserved, bridges avoid)
`TCP 4533`	hackrf-wsjtx-bridge CAT
`TCP 4534`	pluto-rx-bridge CAT (opt-in)
`TCP 4535`	rtlsdr-rx-bridge CAT (opt-in)

Default port	Used by
`TCP 4536`	pluto-wsjtx-bridge CAT

Per-instance: increment by 1 (49813/50005, 4537, 40002, …).

> **Author contact:** Andreas Junge, N6NU <andreas@n6nu.org>. License: GPLv3 for the bridge code, LGPLv2 / LGPLv3 / MIT for bundled third-party libraries (per `THIRD_PARTY_LICENSES.md` shipped inside each installer). Bug reports welcome at andreas@n6nu.org or via the issue tracker on the relevant binary repo above.